Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Ярмолинский И.В.
Проверил:
Крощенко А.А.

Брест 2026

Цель работы: Изучить применение **линейной** и **логистической регрессии** для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты

Ход работы

Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.

Вариант 10

• Регрессия (Прогнозирование счастья в странах)
1. World Happiness Report
2. Предсказать оценку счастья (Score)
3. Задания:
   - загрузите данные. В качестве признаков используйте GDP per capita, Social support, Healthy life expectancy;
   - обучите модель линейной регрессии;
   - рассчитайте MSE и R2;
   - визуализируйте зависимость Score от GDP per capita с линией регрессии.

• Классификация (Прогнозирование оттока клиентов)
1. Telco Customer Churn
2. Предсказать, уйдёт ли клиент (Churn = 'Yes')
3. Задания:
   - загрузите данные, обработайте категориальные признаки;
   - обучите модель логистической регрессии;
   - рассчитайте Accuracy, Precision и Recall для класса 'Yes';
   - постройте матрицу ошибок.

Код программы для первого задания lab2.py:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

```
from sklearn.preprocessing import StandardScaler

def happiness_regression():

    print("\n==================================")
    print("РЕГРЕССИЯ: ПРОГНОЗИРОВАНИЕ СЧАСТЬЯ")
    print("==================================")

    try:
        df = pd.read_csv("data/world_happiness_report.csv")
    except Exception as e:
        print("Ошибка загрузки датасета:", e)
        return

    print("\nПервые строки датасета:")
    print(df.head())

    features = [
        "GDP per capita",
        "Social support",
        "Healthy life expectancy"
    ]

    target = "Score"

    # Проверяем наличие необходимых столбцов
    required_columns = features + [target]

    missing_columns = [
        col for col in required_columns if col not in df.columns
    ]

    if missing_columns:
        print("В датасете отсутствуют столбцы:", missing_columns)
        print("Имеющиеся столбцы:", df.columns.tolist())
        return

    # Удаляем строки с пропущенными значениями
    df = df[required_columns].dropna()

    X = df[features]
    y = df[target]

    # Разделение данных на обучающую и тестовую выборки
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # Создание и обучение модели
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Предсказание
    y_pred = model.predict(X_test)

    # Метрики качества
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
```

```
    print("\nРЕЗУЛЬТАТЫ ЛИНЕЙНОЙ РЕГРЕССИИ")
    print("-----------------------------------")
    print("MSE:", round(mse, 4))
    print("R2:", round(r2, 4))

    print("\nКоэффициенты модели:")
    for name, coef in zip(features, model.coef_):
        print(name, ":", round(coef, 4))

    print("Свободный коэффициент:", round(model.intercept_, 4))

    # Визуализация зависимости Score от GDP per capita
    plt.figure(figsize=(9, 6))

    plt.scatter(
        df["GDP per capita"],
        df["Score"],
        color="blue",
        label="Реальные данные"
    )

    # Строим линию регрессии
    x_line = np.linspace(
        df["GDP per capita"].min(),
        df["GDP per capita"].max(),
        100
    )

    # Остальные признаки фиксируем на средних значениях
    prediction_data = pd.DataFrame({
        "GDP per capita": x_line,
        "Social support": df["Social support"].mean(),
        "Healthy life expectancy":
            df["Healthy life expectancy"].mean()
    })

    y_line = model.predict(prediction_data)

    plt.plot(
        x_line,
        y_line,
        color="red",
        linewidth=2,
        label="Линия регрессии"
    )

    plt.xlabel("GDP per capita")
    plt.ylabel("Score")
    plt.title("Зависимость счастья от ВВП на душу населения")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()


def churn_classification():
```

```
    print("\n==================================")
    print("КЛАССИФИКАЦИЯ: ОТТОК КЛИЕНТОВ")
    print("==================================")

    try:
        df = pd.read_csv("data/Telco-Customer-Churn.csv")
    except Exception as e:
        print("Ошибка загрузки датасета:", e)
        return

    print("\nПервые строки датасета:")
    print(df.head())

    print("\nРазмер датасета:", df.shape)

    # Проверяем наличие необходимых столбцов
    if "customerID" not in df.columns or "Churn" not in df.columns:
        print("В датасете отсутствуют необходимые столбцы customerID
или Churn.")
        print("Имеющиеся столбцы:", df.columns.tolist())
        return

    # Удаляем идентификатор клиента
    df = df.drop(columns=["customerID"])

    # Преобразуем TotalCharges в числовой формат
    df["TotalCharges"] = pd.to_numeric(
        df["TotalCharges"], errors="coerce"
    )

    df = df.dropna()

    # Целевая переменная:
    # Yes = 1 (клиент уйдёт)
    # No = 0 (клиент останется)
    y = (df["Churn"] == "Yes").astype(int)

    X = df.drop(columns=["Churn"])

    # Обработка категориальных признаков
    X = pd.get_dummies(X, drop_first=True)

    # Разделение на обучающую и тестовую выборки
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )

    # Масштабирование признаков
    scaler = StandardScaler()

    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    # Создание и обучение логистической регрессии
```

```
    model = LogisticRegression(max_iter=2000)
    model.fit(X_train, y_train)

    # Предсказание
    y_pred = model.predict(X_test)

    # Метрики качества
    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(
        y_test, y_pred, pos_label=1, zero_division=0
    )

    recall = recall_score(
        y_test, y_pred, pos_label=1, zero_division=0
    )

    print("\nРЕЗУЛЬТАТЫ ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ")
    print("-----------------------------------")

    print("Accuracy:", round(accuracy, 4))
    print("Precision (Yes):", round(precision, 4))
    print("Recall (Yes):", round(recall, 4))

    # Матрица ошибок
    cm = confusion_matrix(y_test, y_pred)

    print("\nМатрица ошибок:")
    print(cm)

    # Визуализация матрицы ошибок
    display = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["No", "Yes"]
    )

    display.plot(cmap="Blues", values_format="d")

    plt.title("Матрица ошибок: прогноз оттока клиентов")
    plt.tight_layout()
    plt.show()

# ==========================================
# ГЛАВНОЕ МЕНЮ
# ==========================================

def main():

    while True:

        print("\n========== ГЛАВНОЕ МЕНЮ ==========")
        print("1. Регрессия (прогнозирование счастья)")
        print("2. Классификация (отток клиентов)")
        print("3. Выполнить обе задачи")
        print("0. Выход")

        choice = input("\nВыберите действие: ")
```

```
        if choice == "1":
            happiness_regression()

        elif choice == "2":
            churn_classification()

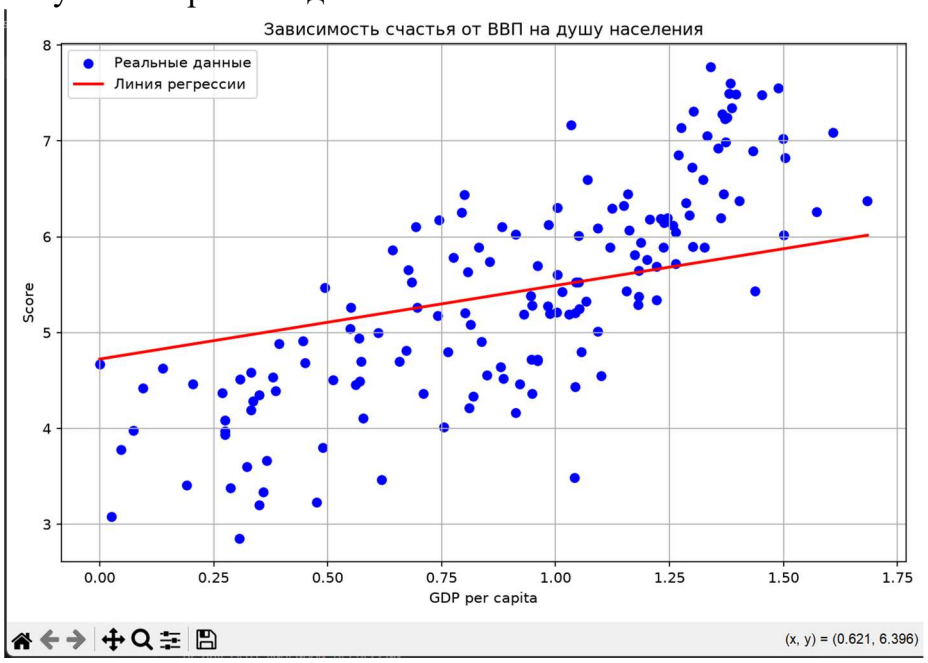
        elif choice == "3":
            happiness_regression()
            churn_classification()

        elif choice == "0":
            print("Программа завершена.")
            break

        else:
            print("Неверный выбор. Попробуйте снова.")

if __name__ == "__main__":
    main()
```

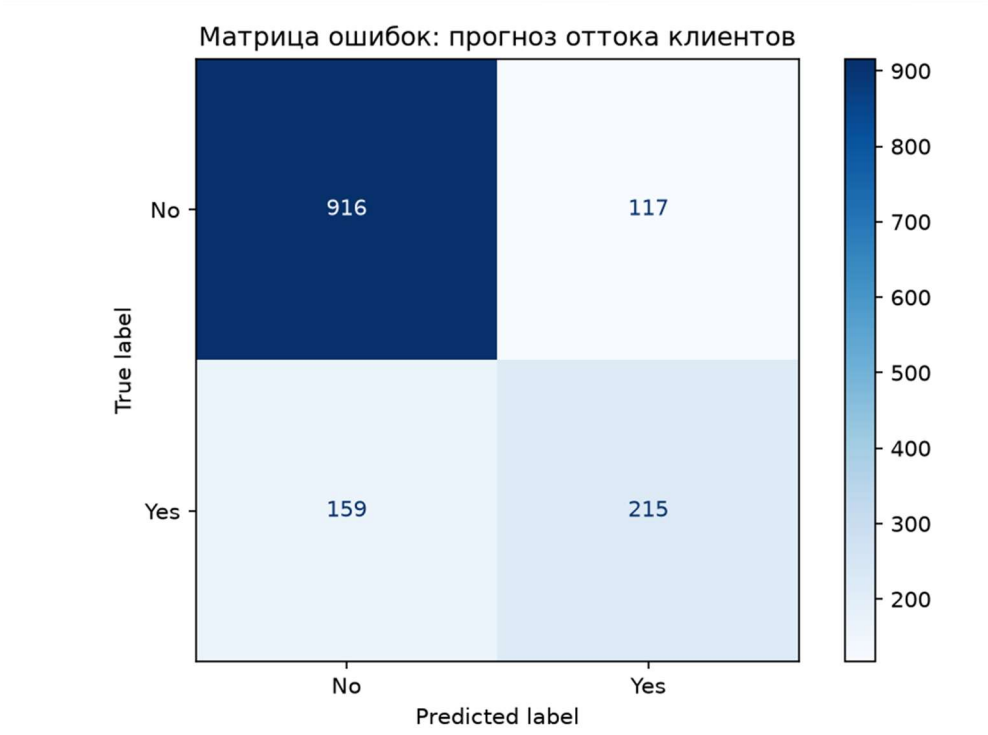
Результат первого задания:



MSE: 0.4637
R2: 0.5545

Коэффициенты модели:
GDP per capita : 0.7675
Social support : 1.5106
Healthy life expectancy : 1.2006
Свободный коэффициент: 2.0265

Результат второго задания



Accuracy: 0.8038

Precision (Yes): 0.6476

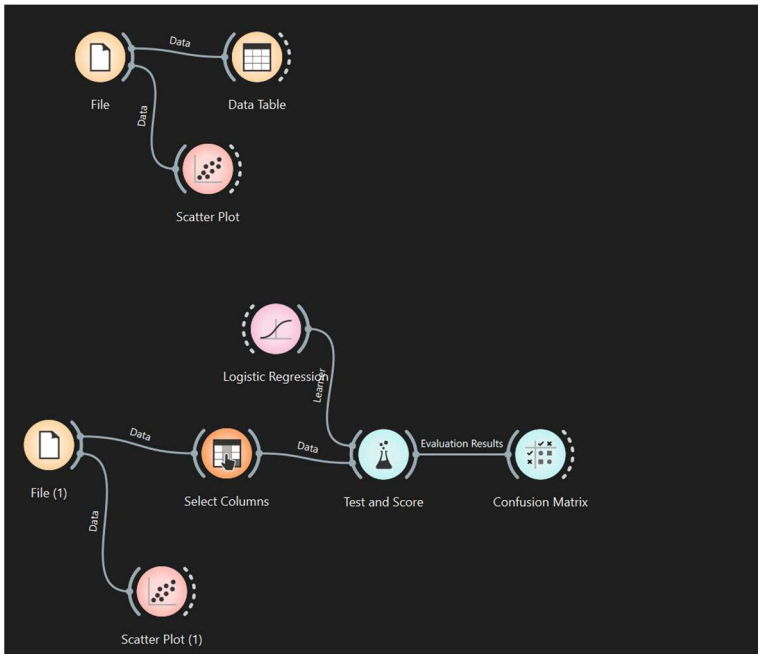
Recall (Yes): 0.5749

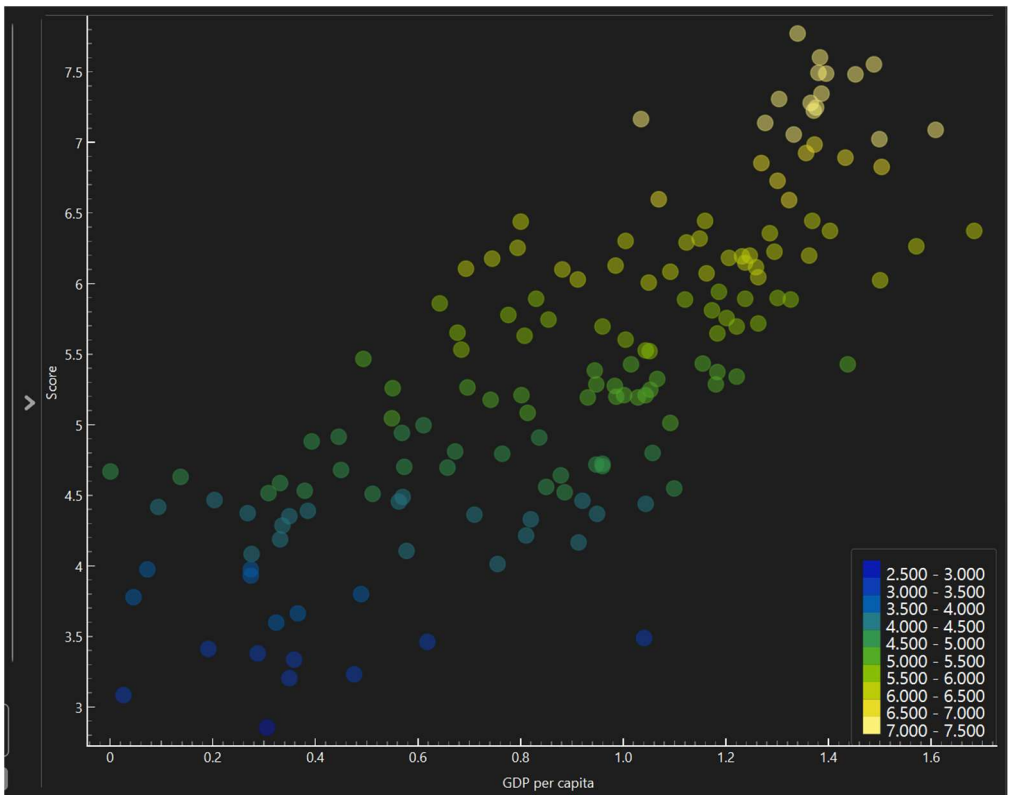
Матрица ошибок:

[[916 117]

 [159 215]]

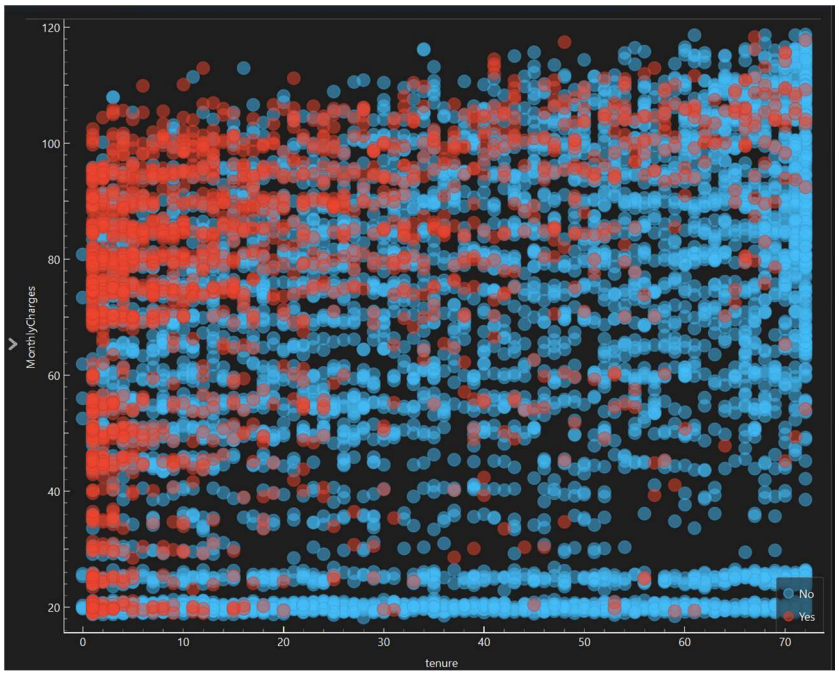
Результаты в Orange:





Show: Number of instances

Actual \ Predicted	No	Yes	Σ
No	4635	539	5174
Yes	838	1031	1869
Σ	5473	1570	7043



Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.